

Miniteste 2 — O simulador

Trabalho individual, para casa

Nome:

Matri-
cula:

Nota. Três das cinco questões são respondidas em português, sem código — valem 65% da nota. Só a Questão 2 pede programação, e são quatro linhas. O objetivo é entender como a máquina funciona, não digitar Python depressa.

Preparação. Baixe `pic18.py` (só precisa de Python 3, nada a instalar):

```
python3 pic18.py programa.s --trace 10 --dump --passos 20
```

Questão 1 (2,5 pontos) Descreva o que a máquina faz

A instrução `incf 0x20,f` (código `0x2A20`) incrementa em um o conteúdo da posição `0x20` da memória.

Descreva, numerado, passo a passo, tudo o que o processador faz para executar essa instrução — do momento em que o contador de programa aponta para ela até o fim. Entre quatro e seis passos. Sem código.

Resposta.

1. **Buscar.** Ler duas palavras de memória de programa no endereço apontado pelo contador de programa, obtendo `0x2A20`.
2. **Avançar o contador** em dois bytes, para a próxima instrução.
3. **Decodificar.** Os bits altos identificam `incf`; o bit *d* indica que o resultado vai para a memória (e não para *W*); o bit *a* indica banco de acesso; os oito bits baixos dão o endereço, `0x20`.
4. **Ler** o conteúdo atual da posição `0x20`.
5. **Somar um**, descartando o que passar de 8 bits.
6. **Escrever o resultado de volta** na posição `0x20`.

Resposta que mencione buscar, decodificar, ler, somar e escrever de volta está completa, mesmo com outras palavras.

Critério. 2,5: os cinco passos essenciais, com leitura e escrita **na memória** explícitas. 1,5: correta, mas o valor passa por um registrador intermediário. 0,5: apenas “soma um na posição `0x20`”. Decisivo: perceber que ler e escrever fazem parte da **mesma** instrução.

Questão 2 (2,0 pontos) Implemente

O simulador não conhece `incf`: para com `??? 0x2A20` (nao implementada). Abra `pic18.py`, ache o comentário `# --- EXERCICIO` dentro de `executa()`, e implemente a instrução — use `decfsz`, logo acima, como modelo (mesma estrutura). Máscara a testar: `0x2800`; formato de `INCF` no datasheet.

Cole abaixo apenas as linhas que você escreveu.

Resposta.

```
if op & 0xFC00 == 0x2800:                                # incf
    v = (self.mem.ler(f, acesso) + 1) & 0xFF
    if d: self.mem.escrever(f, v, acesso)
    else: self.W = v
    return f"incf 0x{f:02X}", 1
```

O `& 0xFF` é obrigatório: a memória tem 8 bits por posição, e `255 + 1` deve dar 0, não 256.

Critério. 2,0: funciona e respeita o bit *d*. 1,5: funciona só para *d* = 1. 1,0: incrementa sem truncar em 8 bits. Não descontar por estilo.

Questão 3 (2,5 pontos) Leia os dois despejos

Os dois programas abaixo diferem em **uma letra**. Foram executados com `--passos 7 --dump`, e os despejos são reais.

Programa A

```
clrf    0x20
movlw   99
incf    0x20,f
incf    0x20,f
incf    0x20,f
fim: bra  fim
```

```
W = 99    ciclos = 9
020: 03 00 00 00 ...
```

Programa B

```
clrf    0x20
movlw   99
incf    0x20,w
incf    0x20,w
incf    0x20,w
fim: bra  fim
```

```
W = 1     ciclos = 9
020: 00 00 00 00 ...
```

Responda em português:

(a) No programa B, por que `0x20` continua valendo zero, mesmo “incrementada” três vezes?

(b) No programa B, por que `W` vale 1 e não 3?

(c) Os dois gastaram exatamente 9 ciclos. O que isso diz sobre a relação entre custo e efeito de uma instrução?

Resposta. (a) O sufixo `,w` põe o bit *d* em zero: o resultado vai para `W`, não volta à memória. A posição `0x20` é lida três vezes e nunca escrita, e permanece com o valor que o `clrf` deixou.

(b) Cada incremento parte do valor lido da memória — sempre zero — e produz 1. As três instruções fazem a mesma coisa: `W` recebe $0 + 1$. Não há acumulação porque o resultado nunca é guardado em lugar que a próxima instrução leia.

(c) O custo em ciclos não depende do destino: as duas versões executam o mesmo número de instruções, no mesmo tempo, com resultados completamente diferentes. Contar ciclos prevê o **tempo**, e nada mais — tempo e correção são propriedades independentes.

Critério. (a): 1,0. (b): 1,0 — o difícil é perceber que não há acumulação; “só executou uma vez” está errada, as três executaram. (c): 0,5, separa nota máxima.

Questão 4 (1,5 ponto) Meça

Rode o programa de piscar fornecido e preencha:

```
python3 pic18.py blink.hex --pino D,0
```

Grandeza	Valor medido
Ciclos entre duas comutações do pino	
Tempo correspondente, em ms (a 250 ns por ciclo)	
Frequência do piscar, em Hz	
Ciclos de uma volta do laço mais interno	

Para a última linha, use `--trace 20` e observe as duas instruções que se repetem.

Resposta.

Grandeza	Valor
Ciclos entre comutações	942 528
Tempo	235,6 ms
Frequência	2,12 Hz
Volta do laço interno	3 ciclos — <code>decfsz</code> (1) mais <code>bra</code> (2)

Aceitar 235,632 ms e 2,1 Hz. A última linha admite a observação de que a **última** volta custa 2, porque o `decfsz` pula o `bra`.

Questão 5 (1,5 ponto) **Conclua**

No Programa A da Questão 3, `0x20` foi incrementada três vezes e terminou valendo 3. `W` terminou valendo **99** — o mesmo valor que o `movlw` havia colocado antes do laço. **Explique por que esse 99 intacto é importante**, e diga o que teria acontecido com `W` num processador ARM executando a mesma tarefa.

Resposta. O **99** intacto mostra que o incremento aconteceu **inteiramente dentro da memória**: leu `0x20`, somou e escreveu de volta, sem usar o acumulador em momento algum.

Num ARM isso é impossível: a arquitetura é *load-store* — instruções aritméticas só operam sobre registradores, e a memória só é alcançada por cargas e armazenamentos. O mesmo incremento exigiria três instruções (carregar, somar, armazenar), e o registrador usado teria perdido o conteúdo anterior.

Na prática, operações sobre **uma** posição de memória — ligar um bit de porta, incrementar um contador — custam uma instrução aqui e três lá. Como é isso que código de controle faz o tempo todo, a diferença aparece.

Critério. 1,5: identifica que a memória mudou sem passar pelo acumulador e que o ARM precisaria de três instruções com registrador intermediário. 1,0: só a primeira parte. 0,5: descreve o 99 sem interpretar. Não exige o termo “load-store”.

Entrega. Este documento respondido, ou as respostas numeradas em folha separada. Prazo: início da aula seguinte.

Atenção. Não existe print no chip. Este trabalho é feito olhando o despejo de memória, porque é assim que se depura um sistema embarcado: sem terminal, sem tela, sem mensagem de erro. Só memória e pinos. Trata-se de uma atividade individual — discutir com colegas é bem-vindo, entregar código idêntico não.